



[CODE](#) > [JSON](#)

Ten una REST API Falsa Funcionando Usando json-server

by [Roy Agasthyan](#) 2 Feb 2017

Difficulty: Beginner Length: Short Languages: Español ▼

[JSON](#)[JavaScript](#)[Web Development](#)

Spanish (Español) translation by [Rafael Chavarría](#) (you can also [view the original English article](#))

En este tutorial, verás como comenzar a configurar y usar un servidor REST API falso usando [json-server](#), el cuál puedes usar mientras desarrollas aplicaciones móviles o web. Este tutorial asume que tienes conocimiento básico de peticiones JSON y HTTP.

¿Qué es REST?

REST significa Representational State Transfer. Es un estilo de arquitectura para diseñar aplicaciones conectadas. Usa HTTP simple para hacer posible comunicación entre máquinas. Así qué, en vez de usar una URL para manipular alguna información de usuario, REST envía una petición HTTP como GET, POST, DELETE, etc. a una URL para manipular datos.

Por ejemplo, en vez de hacer una petición GET a una URL como `/deleteuser?id=10`, la petición sería como `DELETE/user/10`.

¿Por Qué Necesitamos una REST API Falsa?

Las APIs REST forman el back end para aplicaciones móviles y web. Cuando se desarrollan aplicaciones, algunas veces no tendrás las APIs REST listas para ser usadas para propósitos de desarrollo. Para ver la app móvil o web en acción, necesitamos un servidor que arroje algunos datos JSON postizos.

Es ahí cuando una API REST falsa viene a lugar. `json-server` proporciona la funcionalidad para establecer un servidor API REST falso con mínimo esfuerzo.

Advertisement

Comenzando

Para comenzar usando `json-server`, instala el paquete usando Node Package Manager (npm).

```
1 | npm install -g json-server
```

Crea un archivo JSON de ejemplo con algo de datos como tu requerimiento. Por ejemplo, necesito algunos datos JSON con información de usuario como id, nombre, ubicación, etc. Así que crearé un archivo llamado `info.json` con la siguiente información JSON:

```
01 | {
02 |   "users": [{
03 |     "id": 1,
04 |     "name": "roy",
05 |     "location": "india"
06 |   }, {
07 |     "id": 2,
08 |     "name": "sam",
09 |     "location": "wales"
10 |   }]
11 | }
```

Desde la terminal, ejecuta el servidor json con `info.json` como fuente de información y deberías tener la API REST corriendo en <http://localhost:3000>.

```
1 | json-server info.json
```

Probando los Puntos finales de la API REST

Ya que nuestro servidor API REST falso está listo, veamos cómo acceder a la API REST usando un cliente. Estoy usando el cliente Postman REST para hacer las llamadas API.

Petición GET

Comencemos haciendo una petición `GET` a la URL REST. Dentro del archivo JSON, hemos definido un punto final `users` que contiene información relacionada a los usuarios. Al hacer una petición `GET` a la URL <http://localhost:3000/users>, esta debería mostrar la información existente.

```
01 | [
02 |   {
03 |     "id": 1,
04 |     "name": "roy",
05 |     "location": "india"
06 |   },
07 |   {
08 |     "id": 2,
09 |     "name": "sam",
10 |     "location": "wales"
11 |   }
12 | ]
```

Advertisement

Petición POST

Para poder agregar nueva información a la información existente, haremos una petición `POST` a la URL <http://localhost:3000/users>. Aquí está cómo se vería la petición `POST`.

Intenta hacer una petición `GET` y deberías tener la información recién agregada en el archivo `info.json`.

```
01  [
02    {
03      "id": 1,
04      "name": "roy",
05      "location": "india"
06    },
07    {
08      "id": 2,
09      "name": "sam",
10      "location": "wales"
11    },
12    {
13      "name": "ii",
14      "location": "la",
15      "id": 7
16    },
17    {
18      "name": "Shona",
19      "location": "LA",
20      "id": 8
21    },
22    {
23      "name": "Shona",
24      "location": "LA",
25      "id": 9
26    }
27  ]
```

Petición DELETE

Para borrar una entrada de los datos `json.server`, necesitas enviar una petición `DELETE` al punto final de la API con el id de usuario. Por ejemplo, para borrar el usuario con id 1, necesitas mandar una petición `DELETE` al punto final <http://localhost:3000/users/1>. Una vez que está borrado, intenta hacer una petición `GET`, y el usuario con id 1 no debería estar en el JSON devuelto.

Petición PATCH

Para actualizar una entrada existente, necesitas enviar una petición `PATCH` con los detalles que necesitan ser actualizados para esa entrada en particular. Por ejemplo, para actualizar los detalles para el usuario con id 2, enviaremos una petición `PATCH` a la URL <http://localhost:3000/users/2> como se muestra:

Buscando APIs REST json-server

Usando APIs REST `json-server`, puedes buscar a través de información para encontrar datos basados en cierto criterio. Por ejemplo, para encontrar usuarios con un nombre en particular, necesitas enviar una petición GET a la URL de la API REST como se muestra:

Como se ve en la imagen de arriba, enviar una petición GET a la URL <http://localhost:3000/users?name=Shona> devolvería los usuarios con nombre `Shona`. De manera similar, para buscar usuarios con cualquier otro campo, necesitas agregar ese campo a la cadena de consulta.

Para poder ejecutar una búsqueda de texto completo a lo largo del punto final de API REST, necesitas agregar la cadena de búsqueda junto con el parámetro `q` al punto final. Por ejemplo, para poder buscar usuarios con información conteniendo la cadena de búsqueda `s`, la petición se vería como:

Manejando Paginación

Mientras mostramos una cuadrícula paginada de datos, se requeriría traer alguna información basada en paginación. En tales escenarios, `json-server` proporciona la

funcionalidad para paginar la información JSON. Por defecto, la cuenta de información devuelta desde `json-server` es 10. Podemos definir explícitamente este límite usando el parámetro `_limit`.

```
1 | http://localhost:3000/users?_limit=5
```

Una petición GET a la URL de arriba devolvería cinco registros. Ahora, para paginar la información, necesitamos agregar otro parámetro `_page` a la URL. `_page` define la página que necesita ser recuperada al regreso de los datos.

```
1 | http://localhost:3000/users?_limit=5&_page=2
```

Una petición GET a la URL de arriba devolvería la segunda página de la información establecida con cinco registros por página. Al cambiar la variable `_page`, podemos recuperar los registros de página requeridos.

Manejando Acomodo

`json-server` proporciona la funcionalidad para ordenar la información recuperada. Podemos ordenar la información proporcionando el nombre de la columna que necesita ser ordenada y el orden en que la información necesita ser ordenada. Por defecto, la información es ordenada en orden ascendente. Podemos proporcionar el nombre de la columna en el punto final de la URL usando la palabra clave `_sort` y definir el orden usando la palabra clave `_order`. Aquí está una URL de ejemplo:

```
1 | http://localhost:3000/users?_sort=id&_order=DESC
```

La URL de arriba ordenaría la información basado en la columna `Id`, y sería ordenada de manera descendente.

Manejando Operadores

`json-server` también proporciona la funcionalidad para soportar operadores como encontrar una entrada con `Id` en un rango entre dos valores o una entrada empatando a una expresión regular particular.

Para encontrar una entrada dentro de un rango particular, podemos hacer uso de los operadores `_gte` y `_lte`. Por ejemplo, para encontrar usuarios con un `id` mayor que 1 y menor que 2, haz una petición GET a la URL http://localhost:3000/users?id_gte=1&id_lte=2 como se muestra:

Si queremos buscar nombres comenzando con cierta letra, podemos usar expresiones regulares. Por ejemplo, para buscar nombres comenzando con letras `sa`, hacemos uso del operador `_like` y hacemos una petición `GET` a la URL http://localhost:3000/users?name_like=^sa.

Conclusión

En este tutorial, viste como usar APIs REST `json-server` para crear una base de datos dummy para uso rápido. Aprendiste cómo comenzar usando `json-server` y consultar la URL para agregar, actualizar, modificar y borrar datos. Viste como paginar, acomodar y buscar la información de prueba. También viste cómo usar operadores para buscar entradas usando expresiones regulares.

¿Has usado `json-server` o cualquier otro servidor API REST falso para creación de datos de prueba? ¿Cómo fue tu experiencia? Déjanos saber tus ideas en los comentarios de abajo.



Roy
Agasthyan

Roy Agasthyan

I'm a software engineer by profession and love to write tutorials and educational stuff.

 FEED

 LIKE

 FOLLOW

Weekly email summary

Subscribe below and we'll send you a weekly email summary of all new Code tutorials. Never miss out on learning about the next big thing.

Update me weekly

Translations

Envato Tuts+ tutorials are translated into other languages by our community members—you can be involved too!

Translate this post

Powered by



Advertisement

QUICK LINKS - Explore popular categories

ENVATO TUTORIALS+



JOIN OUR COMMUNITY



HELP



 envato-tutorials

28,829
Tutorials

1,282
Courses

41,126
Translations

© 2020 Envato Pty Ltd. Names and brands are the property of their respective owners.

Follow Envato Tuts+



Facebook



Twitter



Pinterest